

Ornith-1.0: самообучающиеся языковые модели для агентного программирования

📅 Июнь 2026

🐦 Twitter

🤗 Hugging Face

Ornith-1.0

Open-Source LLMs
Specialized for Agentic Coding

✓ 9b-dense ✓ 31b-dense
✓ 35b-moe ✓ 397b-moe



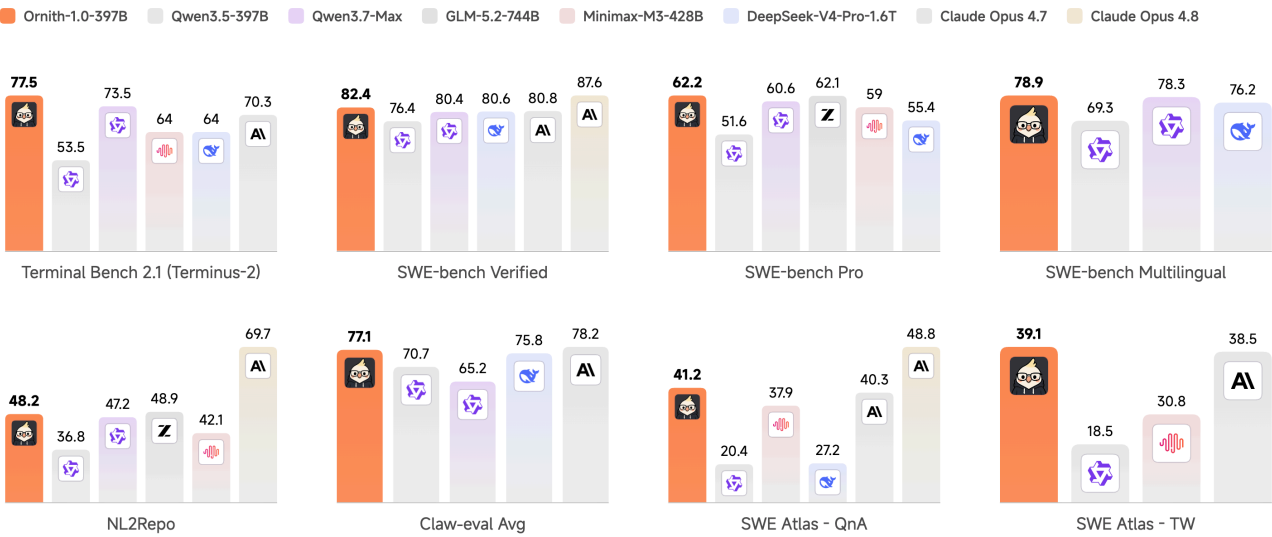
Алоха! 🌺

Сегодня мы представляем **Ornith-1.0** — самосовершенствующееся семейство моделей с открытым исходным кодом, разработанных специально для задач агентного программирования. Ornith-1.0 охватывает весь спектр — от компактных моделей 9B Dense, подходящих для периферийных устройств, до передовых моделей 397B MoE, оптимизированных для максимальной производительности, с вариантами **9B Dense, 31B Dense, 35B MoE и 397B MoE**. Созданная на основе предварительно обученных моделей Gemma 4 и Qwen 3.5, она демонстрирует самую высокую производительность среди моделей с открытым исходным кодом сопоставимого размера в тестах на кодирование.

Ключевое нововведение Ornith-1.0 — это самосовершенствующаяся система обучения. Вместо того чтобы полагаться на разработанные людьми «обязки» для генерации решений в рамках обучения с подкреплением, Ornith-1.0 учится генерировать как варианты решения, так и «обязки» для конкретных задач, которые направляют этот процесс. Благодаря совместной оптимизации «обязки» и получаемого решения модель может находить более эффективные траектории поиска и генерировать решения более высокого качества.

Ornith-1.0 обеспечивает высочайшую производительность среди моделей с открытым исходным кодом сопоставимого размера в широком диапазоне тестов агентного кодирования: Ornith-1.0-397B (**77,5** на терминале 2.1 и **82,4** на SWE-Bench Verified) соответствует производительности Claude Opus 4.7 (**70,3** на TB-2.1 и **80,8** на SWE-Bench Verified) и превосходит ведущие модели с открытым исходным кодом аналогичного размера, включая MiniMax M3 (**66,0** на TB-2.1 и **80,5** на SWE-Bench Verified). Проверено на SWE-Bench) и DeepSeek-V4-Pro (**67,9** на TB-2.1 и **80,6** на SWE-Bench проверено). Ornith-1.0-9B, который можно легко развернуть на периферийных устройствах, по производительности не уступает или даже превосходит гораздо более крупные модели, такие как Gemma 4-31B и Qwen 3.6 35B.

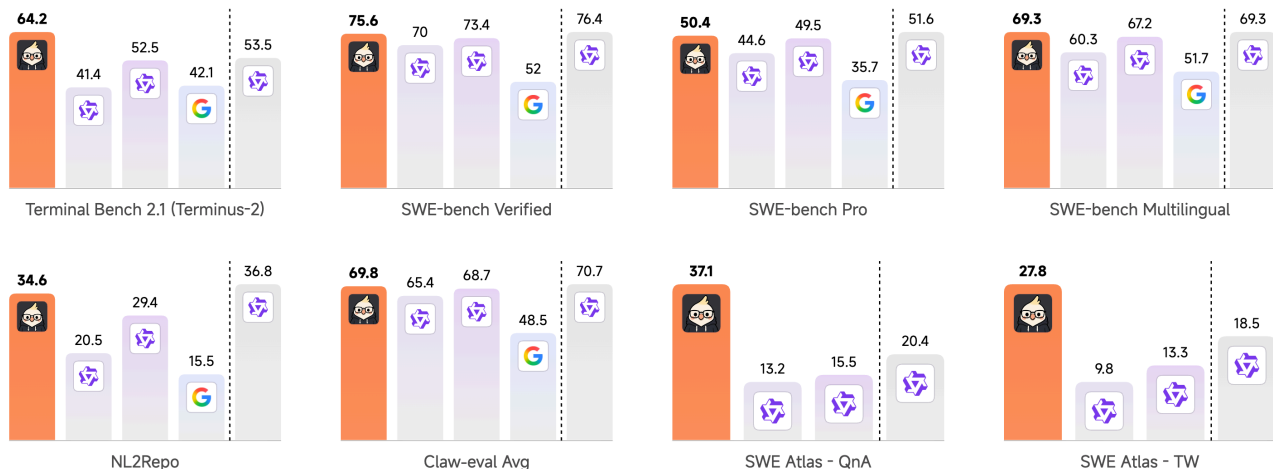
LLM Performance Evaluation



В флагманском масштабе Ornith-1.0-397B показывает результаты **77,5** на Terminal-Bench 2.1 и **82,4** на SWE-Bench Verified, превосходя Claude Opus 4.7 по обоим показателям и лидирующие модели с открытым исходным кодом аналогичного размера, включая Minimax M3 и DeepSeek-V4-Pro.

LLM Performance Evaluation

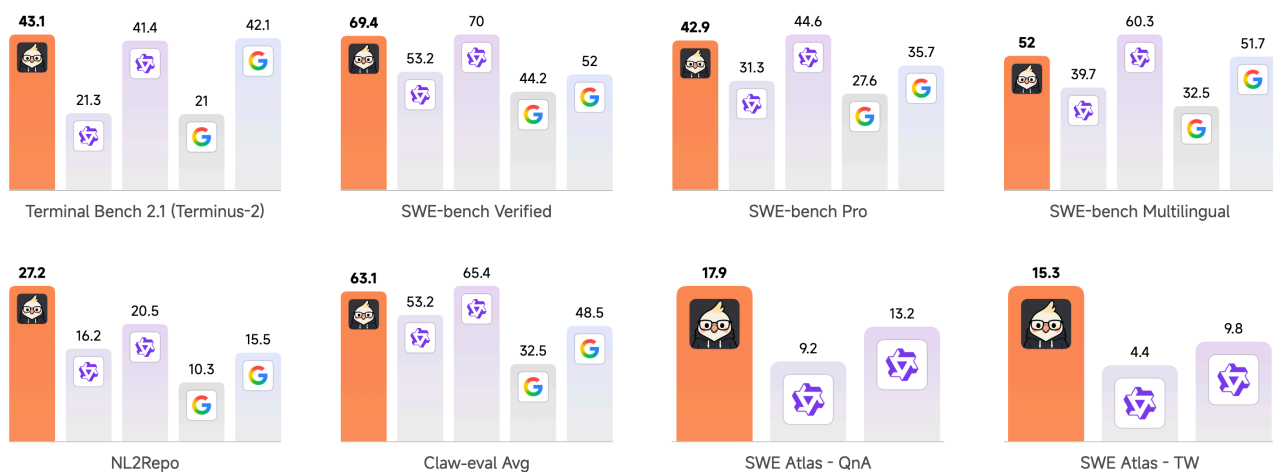
Ornith-1.0-35B Qwen3.5-35B Qwen3.6-35B Gemma4-31B Qwen3.5-397B



Ornith-1.0-35B значительно превосходит модели аналогичного размера, в том числе Qwen 3.5-35B, Qwen 3.6-35B и Gemma 31B. Несмотря на то, что у него всего 35 миллиардов параметров, он даже превосходит Qwen 3.5-397B в тесте Terminal-Bench 2.1 (64,4 против 53,5), а по некоторым другим тестам на кодирование и агентное моделирование не уступает ему.

LLM Performance Evaluation

Ornith-1.0-9B Qwen3.5-9B Qwen3.5-35B Gemma4-12B Gemma4-31B



Модель Ornith-1.0-9B, развертываемая на периферии, также демонстрирует впечатляющие результаты, набрав **43,1** балла на Terminal-Bench 2.1 и **69,4** балла на SWE-Bench Verified. Несмотря на то, что это компактная модель с 9 миллиардами параметров, она не уступает по производительности гораздо более крупным моделям, таким как

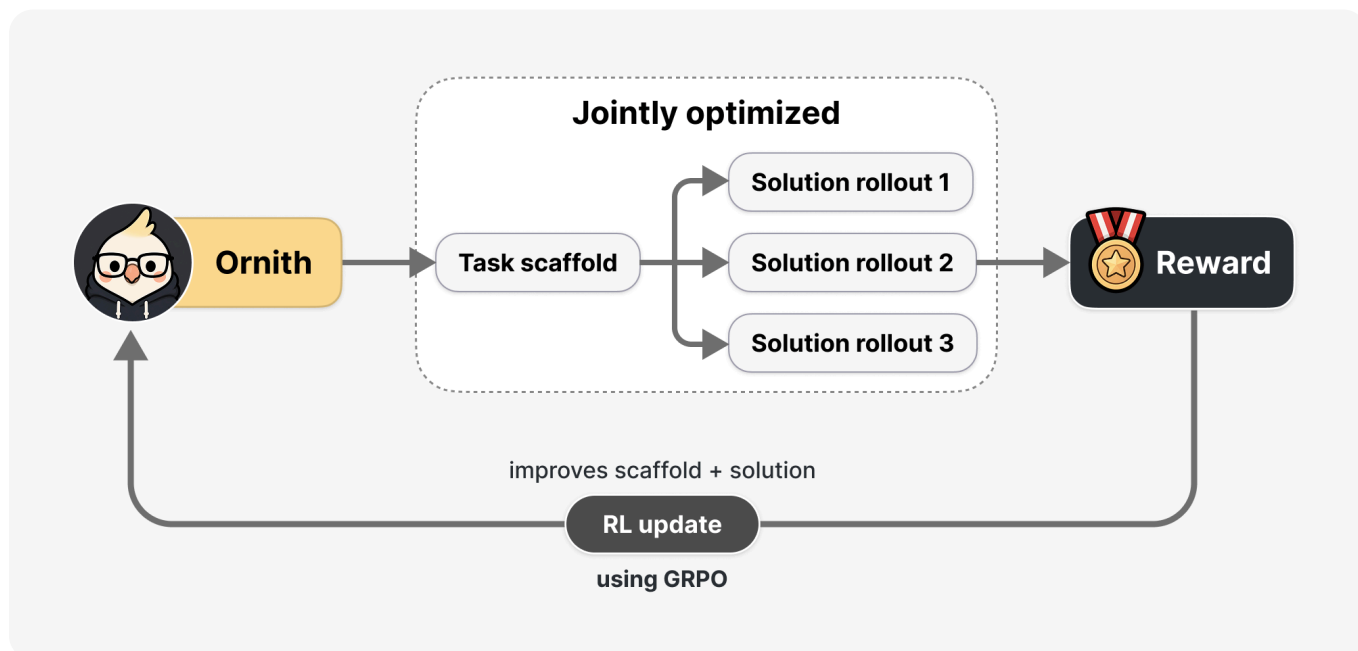
Gemma 4-31B, и даже превосходит их, демонстрируя, что даже при ограниченном количестве ресурсов можно добиться высокой эффективности агентного кодирования.

Самосовершенствующаяся стратегия обучения больших языковых моделей

В основе Ornith-1.0 лежит самосовершенствующаяся система обучения, которая совместно с моделью обучается решать задачи и создавать структуры, на основе которых эти решения принимаются. Вместо того чтобы полагаться на фиксированную, разработанную человеком структуру, общую для всей категории задач, Ornith-1.0 рассматривает структуру как обучаемый объект, который развивается вместе с моделью.

Каждый шаг в процессе обучения с подкреплением состоит из двух этапов: на основе задачи и ранее использованного шаблона модель сначала предлагает усовершенствованный шаблон, а затем, на основе этого шаблона и описания задачи, генерирует развертывание решения. Вознаграждение за развертывание решения распространяется на оба этапа, поэтому модель оптимизируется не только для того, чтобы давать более качественные ответы, но и для того, чтобы создавать сценарии, которые приводят к этим ответам.

При многократном повторении этого процесса возникает петля обратной связи, в которой каркасы постоянно видоизменяются и отбираются в пользу тех, которые обеспечивают более высокие показатели вознаграждения. Это позволяет стратегиям для каждой категории задач формироваться автоматически и обеспечивает устойчивый рост возможностей без разработки «обмундирования» вручную.



Решение проблемы «взлома вознаграждения» при самосовершенствовании

Если позволить модели создавать собственный каркас, то естественным образом возникнет проблема «взлома вознаграждения». Самостоятельно сгенерированный каркас может научиться удовлетворять требованиям верификатора, не выполняя задачу: он может считывать видимые тестовые файлы и жестко прописывать ожидаемые артефакты, например изменять проверяемый файл, записывать буквальный ожидаемый результат или копировать решение-оракул, присутствующее в среде.

Мы защищаемся от этого на трех уровнях. Во-первых, мы фиксируем внешнюю границу доверия: окружающая среда, интерфейс инструмента и тестовая изоляция неизменны и находятся вне досягаемости модели, поэтому модель развивает только внутреннюю структуру политики: ее память, обработку ошибок и логику оркестрации.

Во-вторых, детерминированный монитор обеспечивает соблюдение этой границы на том уровне, который можно точно указать, отмечая любые попытки прочесть закрытые пути, изменить проверочные скрипты или выполнить действия за пределами разрешенной области применения инструмента, и присваивая таким траекториям нулевое вознаграждение с исключением из расчета преимущества.

В-третьих, поскольку манипулирование на уровне намерений может происходить исключительно в рамках разрешенных инструментов, замороженный судья на основе больших языковых моделей выступает в роли вето, накладываемого поверх верификатора, а не в качестве основной награды.

Асинхронное обучение с подкреплением

Для обучения с подкреплением, чтобы решить проблему автономной политики при длительных развертываниях, в Ornith-1.0 используется стратегия конвейерного обучения с подкреплением. Чтобы контролировать влияние ранее сгенерированных токенов, мы применяем коэффициент устаревания $w(d_t)$ that downweights tokens according to their age d_t and drops them entirely once a threshold is exceeded:

$$w(d_t) = \begin{cases} 1, & \text{if } d_t \leq K_1, \\ \exp(-\lambda(d_t - K_1)), & \text{if } K_1 < d_t \leq K_2, \\ 0, & \text{if } d_t > K_2. \end{cases}$$

The token-level GRPO loss is weighted as follows:

$$L_t = \min(r_t A_t, \text{clip}(r_t, 1 - \epsilon^-, 1 + \epsilon^+) A_t) \cdot w(d_t),$$

where

$$r_t = \frac{\pi_{\theta}(y_t \mid x, y_{<t})}{\pi_{\theta^{\text{beh}}}(y_t \mid x, y_{<t})}$$

Full Table

Benchmark	Ornith-1.0-397B	Qwen3.5-397B	Qwen3.7-Max	GLM-5.2-744B	Minimax-M3-428B	DeepSeek-V4-Pro-1.6T
Terminal Bench 2.1 (Terminus-2)	77.5	53.5	73.5	81.0	64	64
Terminal Bench 2.1 (Claude Code)	78.2	48.6	69.8	82.7	-	66.5
SWE-Bench Verified	82.4	76.4	80.4	-	-	80.6
SWE-Bench Pro	62.2	51.6	60.6	62.1	59	55.4
SWE-Bench Multilingual	78.9	69.3	78.3	-	-	76.2

Benchmark	Ornith-1.0-397B	Qwen3.5-397B	Qwen3.7-Max	GLM-5.2-744B	Minimax-M3-428B	DeepSeek-V4-Pro-1.6T
NL2Repo	48.2	36.8	47.2	48.9	42.1	-
ClawEval Avg	77.1	70.7	65.2	-	-	75.8
SWE Atlas - QnA	41.2	20.4	-	-	37.9	27.2
SWE Atlas - RF	42.6	18.4	-	-	-	-
SWE Atlas - TW	39.1	18.5	-	-	30.8	-

Benchmark	Ornith-1.0-35B	Qwen3.5-35B	Qwen3.6-35B	Gemma4-31B
Terminal Bench 2.1 (Terminus-2)	64.2	41.4	52.5	42.1
Terminal Bench 2.1 (Claude Code)	62.8	38.9	49.2	-
SWE-Bench Verified	75.6	70	73.4	52
SWE-Bench Pro	50.4	44.6	49.5	35.7
SWE-Bench Multilingual	69.3	60.3	67.2	51.7
NL2Repo	34.6	20.5	29.4	15.5
ClawEval Avg	69.8	65.4	68.7	48.5
SWE Atlas - QnA	37.1	13.2	15.5	-
SWE Atlas - RF	29.7	10.2	11.4	-
SWE Atlas - TW	27.8	9.8	13.3	-

Benchmark	Ornith-1.0-9B	Qwen3.5-9B	Qwen3.5-35B	Gemma4-12B
Terminal Bench 2.1 (Terminus-2)	43.1	21.3	41.4	21
Terminal Bench 2.1 (Claude Code)	40.6	18.9	38.9	-
SWE-Bench Verified	69.4	53.2	70	44.2
SWE-Bench Pro	42.9	31.3	44.6	27.6
SWE-Bench Multilingual	52	39.7	60.3	32.5
NL2Repo	27.2	16.2	20.5	10.3
ClawEval Avg	63.1	53.2	65.4	32.5
SWE Atlas - QnA	17.9	9.2	13.2	-
SWE Atlas - RF	16.6	4.3	10.2	-
SWE Atlas - TW	15.3	4.4	9.8	-

Footnote

- Terminal-Bench 2.1 (Terminus-2): We evaluate Terminal-Bench 2.1 using the Harbor/Terminus-2 framework with parser=json, temperature=1.0, top_p=1.0, and a 128K context window. Each run uses a 4-hour timeout with 32 CPU cores and 48GB RAM, and results are averaged over 5 runs. We adjust the Qwen chat template to ensure consistency between training and inference (https://huggingface.co/deepreinforce-ai/Ornith-1.0-397B/blob/main/chat_template.jinja), and modify Harbor to align with vLLM's reasoning_content key.
- Terminal-Bench 2.1 (Claude Code): We evaluate Terminal-Bench 2.1 using Claude Code 2.1.126 with parser=json, temperature=1.0, top_p=1.0, max_new_tokens=131072. Results are averaged over 5 runs. Again, Qwen chat template needs to be modified.
- SWE-Bench Verified, Pro and Multilingual: using OpenHands harness with temp=1.0, top_p=0.95, 256k context window.

- SWE Atlas QnA, RF, TW: using mini SWE agent harness with temp=1.0, top_p=0.95, 128K context window. Results are averaged over 5 runs.
- NL2Repo: with temperature=1.0, top_p=1.0, 400K context, 48K output and anti-hacking filters.
- ClawEval: An agentic code benchmark over real-user task distributions; temp=0.6 and 256K context.



Ornith

© 2026 DeepReinforce.AI Team. All rights reserved.